

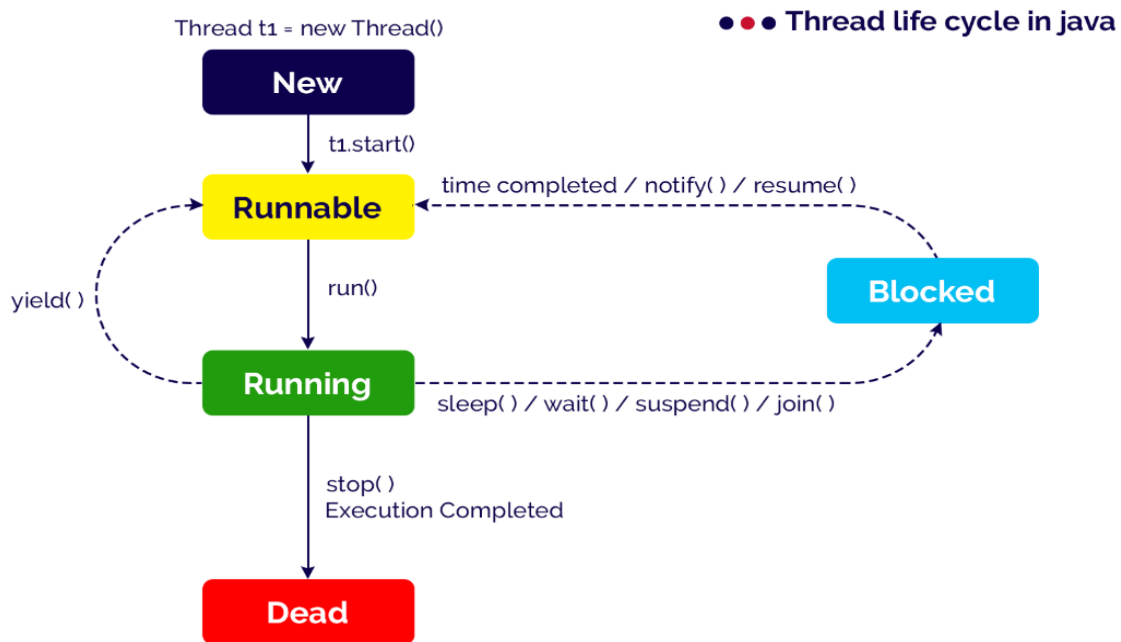
Thread

एक थ्रेड (Thread) किसी **प्रोसेस (Process)** के भीतर निष्पादन (Execution) की सबसे छोटी इकाई होती है। इसे **"लाइटवेट सब-प्रोसेस"** भी कहा जाता है क्योंकि यह एक अलग रास्ता होता है जिससे प्रोग्राम का कोड रन होता है।

Definition (परिभाषा): Java में **Thread** एक प्रोग्राम के अंदर का सबसे छोटा हिस्सा (Smallest unit) होता है जो स्वतंत्र रूप से चल सकता है। एक भारी प्रोग्राम को छोटे-छोटे टुकड़ों में बाँटना और उन्हें एक साथ चलाना ही **Multithreading** कहलाता है।

सरल शब्दों में: जैसे एक वेब ब्राउज़र में एक साथ गाना बजना, फाइल डाउनलोड होना और टाइपिंग करना, ये सब अलग-अलग थ्रेड्स का काम है।

Thread Life Cycle (The 5 States)



Java में थ्रेड की अवस्थाएं Thread.State enum द्वारा परिभाषित की जाती हैं:

1. **New (Born):** Thread t = new Thread(); - यहाँ ऑब्जेक्ट बन गया है लेकिन एक्सीक्यूशन शुरू नहीं हुआ है।
2. **Runnable (Ready):** t.start(); - थ्रेड "Thread Scheduler" के पास चला गया है। यह चलने के लिए तैयार है।
3. **Running:** जब Scheduler थ्रेड को CPU टाइम देता है, तो वह Running स्टेट में आता है और run() मेथड एक्सीक्यूट होता है।
4. **Timed Waiting / Blocked:** जब थ्रेड sleep(ms), wait(), या किसी resource (जैसे I/O) का इंतज़ार कर रहा हो।
5. **Terminated (Dead):** जब run() मेथड पूरी तरह समाप्त हो जाता है या कोई अनहैंडल्ड एरर आ जाती है।

3. Creating Threads (Syntaxes)

Method 1: Extending the Thread Class

इसका उपयोग तब करें जब आप Thread क्लास के व्यवहार को बदलना चाहते हैं।

```
class MyProcessor extends Thread {
    public void run() {
```

```

        System.out.println("Thread ID: " + Thread.currentThread().getId() + " is running.");
    }
}

```

Method 2: Implementing the Runnable Interface

यह सबसे अच्छा तरीका है (Best Practice) क्योंकि यह आपको किसी दूसरी क्लास को भी extend करने की अनुमति देता है।

```

class MyTask implements Runnable {
    public void run() {
        System.out.println("Task execution started.");
    }
}

```

5. Java में Thread कैसे बनाते हैं? (Basic Creation)

Thread class को extend करके

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Test {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}

```

4. Using and Controlling Threads

Action	Method Name	Description
Start	start()	थ्रेड को रजिस्टर करता है और run() को शुरू करता है।
Pause	Thread.sleep(ms)	थ्रेड को कुछ मिलीसेकंड के लिए रोकता है।
Join	join()	करंट थ्रेड को तब तक रोकता है जब तक कि दूसरा थ्रेड खत्म न हो जाए।
Interrupt	interrupt()	थ्रेड को संकेत देता है कि उसे अपना काम बंद कर देना चाहिए।

Non-Thread Program (Single Thread Program)

Example:

```

class Test {
    public static void main(String[] args) {

        for(int i=1; i<=4; i++){
            System.out.println("Task 1: " + i);
        }

        for(int i=1; i<=4; i++){
            System.out.println("Task 2: " + i);
        }
    }
}

```

Output:

```
Task 1: 1
Task 1: 2
Task 1: 3
Task 1: 4
Task 2: 1
Task 2: 2
Task 2: 3
Task 2: 4
```

Explanation:

- यह **single thread program** है
- Task 1 पूरा होने के बाद ही Task 2 शुरू होगा
- कोई भी काम parallel नहीं हो रहा

6. Multithreading Program (Thread वाला Program)

Example: दो threads एक साथ चल रहे हैं

```
class MyThread1 extends Thread {
    public void run() {
        for(int i=1; i<=5; i++){
            System.out.println("Task 1: " + i);
        }
    }
}

class MyThread2 extends Thread {
    public void run() {
        for(int i=1; i<=5; i++){
            System.out.println("Task 2: " + i);
        }
    }
}

public class Test {
    public static void main(String[] args) {

        MyThread1 t1 = new MyThread1();
        MyThread2 t2 = new MyThread2();

        t1.start(); // new thread created
        t2.start(); // another new thread created
    }
}
```

Possible Output:

```
Task 1: 1
Task 1: 2
Task 2: 1
Task 1: 3
Task 1: 4
Task 2: 2
Task 2: 3
Task 2: 4
Task 2: 5
Task 1: 5
```

Explanation:

- यहाँ **दो threads** हैं
- दोनों CPU को share कर रहे हैं
- Output का order fix नहीं होता
- दोनों tasks **simultaneously execute** होते हैं

7. Single Thread vs Multithreading

Single Thread	Multithreading
एक ही task एक समय पर	Multiple tasks एक साथ
Slow execution	Fast execution
Blocking problem	Better performance
Simple programs	Games, servers, apps

5. Thread Exceptions and Stopping

A. Thread Exceptions

सबसे प्रमुख एक्सेप्शन **InterruptedException** है। यह एक **Checked Exception** है।

- **Reason:** जब कोई थ्रेड सोया हुआ (sleep) हो और हम उसे `interrupt()` कर दें, तो यह एरर आता है।

B. Stopping a Thread

Java में `stop()` मेथड **Deprecated** (असुरक्षित) है क्योंकि यह डेटा को कर्प्ट कर सकता है। थ्रेड को रोकने के लिए हमेशा एक "Interrupt" या "Volatile Boolean" का उपयोग करें।

Key Takeaways (महत्वपूर्ण बातें)

- **Main Thread:** हर जावा प्रोग्राम कम से कम एक थ्रेड (main) पर चलता है।
- **Daemon Threads:** ये बैकग्राउंड थ्रेड होते हैं (जैसे Garbage Collector) जो प्रोग्राम के खत्म होते ही अपने आप बंद हो जाते हैं।
- **Concurrency vs Parallelism:** एक CPU कोर पर थ्रेड्स के बीच स्विच करना Concurrency है, जबकि मल्टी-कोर CPU पर एक साथ थ्रेड्स चलाना Parallelism है।

Next Step: क्या आप **Thread Synchronization** (ताकि दो थ्रेड्स एक साथ एक ही वेरिएबल को खराब न करें) सीखना चाहेंगे या फिर **Exception Handling** के throw और throws की बारीकियां देखें?